Chapter 6 Temporal-Difference Learning

本章介绍了**时序差分学习 (Temporal-Difference learning, 简称 TD 学习)**，这是一种结合了**动态规划 (Dynamic Programming, DP)** 和**蒙特卡洛方法 (Monte Carlo methods, MC)** 思想的新型强化学习方法。

TD 学习的强大之处在于：它可以直接从经验中学习，而不需要环境的动态模型；并且它可以基于当前已有的价值估计来更新新的价值估计，而无需等到一个回合（episode）完全结束（这意味着它可以进行**自举/bootstrapping**）。之前的用MC得跑完一个start-end的完整循环，太耗时间了。

对于**控制问题 (Control problem)**（即寻找最优策略），动态规划 (DP)、时序差分 (TD) 和蒙特卡洛 (MC) 方法都使用了某种形式的**广义策略迭代 (Generalized Policy Iteration, GPI)**。这些方法的核心差异主要体现在它们处理**预测问题 (Prediction problem)**（即估计状态价值）的方式上。

GPI说白了也就是：1.策略评估；2.策略改进 最后的名词解释有详细介绍。

TD 预测 (TD Prediction)

对于之前用蒙特卡洛预测的review：

给定一个策略 π，我们可以使用蒙特卡洛方法来估计状态的价值。具体做法是：模拟遵循策略 π 的完整回合 (episodes)，对于每一个访问到的状态 s_t，使用以下公式更新其价值估计 $V(s_t)$：

$$V(s_t) \leftarrow V(s_t) + \alpha \left(g_t - V(s_t)\right),$$

这里的 g_t 是在回合结束时计算出的真实累积回报。因为必须等到回合结束才能得到 g_t 并进行更新，如果回合非常长，这种方法的效率会非常低。

公式解释换句话说：新预估分数 = 老预估分数 + 学习率 $\times$ (实际得分 - 老预估分数)。

公式只负责干一件事：给现有的固定策略打分（策略评估），它根本没打算去进化策略！所以没有Q 和 π。

打个比方：

假设你正在练习打一个很难的 Boss，你现在的策略 π 是固定的（比如只会无脑平 A）。
我们设定学习率 $\alpha = 0.1$。
游戏里有两个关键状态：

- S_1： 刚进 Boss 房间，Boss 满血。
- S_2： Boss 剩一半血，进入狂暴二阶段。

初始状态（还没开始玩）：
你的心里没底，预估分全为 0：$V(S_1) = 0, V(S_2) = 0$。

🎮 第 1 局 (Episode 1)

你按策略无脑平 A，经历了 S_1，经历了 S_2，最后居然**通关了**，总奖励 100 分。

- **对于 S_1：** 实际拿到 $g = 100$。

$$V(S_1) \leftarrow 0 + 0.1 \times (100 - 0) = 10$$

（更新后，你觉得开局状态值 10 分了）

- **对于 S_2：** 实际拿到 $g = 100$。

$$V(S_2) \leftarrow 0 + 0.1 \times (100 - 0) = 10$$

🎮 第 2 局 (Episode 2)

你又按策略打，经历了 S_1 和 S_2，但这次运气不好，在二阶段被 Boss **秒杀了**，总奖励 -50 分。

- **对于 S_1：** 实际拿到 $g = -50$。

$$V(S_1) \leftarrow 10 + 0.1 \times (-50 - 10) = 10 - 6 = 4$$

（被坑了一次，评分从 10 跌到了 4）

- **对于 S_2：** 实际拿到 $g = -50$。

$$V(S_2) \leftarrow 10 + 0.1 \times (-50 - 10) = 4$$

🎮 第 N 局 (Episode N)

随着你玩了 100 局、1000 局，经历了无数次的通关（+100）和阵亡（-50）。
每次拿到真实的 g_t，你都用这个公式把 $V(S)$ 往真实分数拉扯一点点（每次拉扯 10%）。
最终结果（获得期望）： $V(S_1)$ 和 $V(S_2)$ 的数值会趋于稳定。比如最终稳定在 30 分。这个 30 分，就是在这个无脑平 A 的策略下，状态 S 的**真实数学期望 V**。

为什么没有 Q？

如果我的目的仅仅是想知道“我现在的打法有多牛逼/多菜”，我只需要评估状态的整体情况 $V(s)$ 就够了。评估 $V(s)$ 是不需要做决策的，自然就不需要算 Q。

如果要算 Q 咋办？

很简单，把公式里的主角换掉就行了！如果你想进入**控制环节（寻求最优解）**，公式就会变成评估状态-动作对：$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(g_t - Q(s_t, a_t))$

策略 π 咋更新？

等上面这个 Q 值的公式跑完，拿着更新好的 Q 表，用之前的 **ϵ-贪心法** 去更新动作概率，策略 π 就完成进化了。

诶，痛点来了。

单步时序差分 (One-step TD / TD(0))

TD 方法不需要等待回合结束，它只需要等待**下一个时间步 (next time step)**。在时间步 $t+1$，TD 方法将目标累积回报 g_t 近似替换为 $r_{t+1} + \gamma V(s_{t+1})$。

$$V(s_t) \leftarrow V(s_t) + \alpha\left(r_{t+1} + \gamma V(s_{t+1}) - V(s_t)\right).$$

这种更新方式被称为 **单步 TD (one-step TD)** 或 **TD(0)**。在 TD 方法中，价值的更新部分依赖于现有的价值估计，这种机制被称为**自举 (bootstrapping)**

结论： TD 结合了蒙特卡洛方法的 **采样 (sampling)** 特性（通过实际交互获得 r_{t+1}）和动态规划的 **自举 (bootstrapping)** 特性（利用后续状态的估计值 $V(s_{t+1})$）。

另外需要注意的是，TD 方法是基于**单一采样状态转移 (one sample transition)** 来更新价值估计的。我们将这种方式称为**采样更新 (sample updates)**。与之相对，动态规划 (DP) 则是基于计算期望结果来更新价值估计的，被称为**期望更新 (expected updates)**。对于任何给定的策略 π，$TD(0)$ 已经被数学证明最终会收敛到真实的价值函数 v_π。

Prediction Problem (预测问题): 在给定一个特定的策略 π 的情况下，计算该策略下每个状态的价值函数 $V^\pi(s)$。也就是评估一个策略有多好。

Bootstrapping (自举): “用自己的鞋带把自己提起来”。在强化学习中，指的是**使用当前的估计值去更新另一个估计值**。TD 和 DP 都使用了自举（用 $V(s_{t+1})$ 更新 $V(s_t)$），而 MC 不使用（MC 必须使用真实的最终结果 g_t）。

TD 误差 (TD errors)

在 TD 更新中，新的近似回报（即 TD 目标）与当前估计值 $V(s_t)$ 之间的差值被称为 **TD 误差 (TD error)**。其正式定义如：

$$\delta_t \doteq r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$$

蒙特卡洛误差与 TD 误差的关系：

需要注意的是，如果在整个回合（episode）中，价值数组 V 保持不变（就像在蒙特卡洛方法中那样），那么**蒙特卡洛误差 (Monte Carlo error)**可以被展开并改写为：

$$
\begin{aligned}
g_t - V(s_t) &= r_{t+1} + \gamma g_{t+1} - V(s_t) \\
&= r_{t+1} + \gamma V(s_{t+1}) - V(s_t) + \gamma g_{t+1} - \gamma V(s_{t+1}) \\
&= \delta_t + \gamma(g_{t+1} - V(s_{t+1})) \\
&= \delta_t + \gamma(\delta_{t+1} + \gamma(g_{t+2} - V(s_{t+2}))) \\
&= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} + \cdots + \gamma^{T-1-t}(g_{T-1} - V(s_{T-1})) \\
&= \sum_{k=t}^{T-1} \gamma^{k-t}\delta_k \quad \text{（公式 6.3）}
\end{aligned}
$$

- δ_t **(小写 Delta)**: 代表时间步 t 时的 TD 误差。
- $\doteq$: 定义为（is defined as）。
- T: 终端时间步 (Terminal time step)，即整个回合结束的时刻。
- $\sum_{k=t}^{T-1}$: 求和符号，表示从当前时间步 t 一直加到回合结束的前一步 $T-1$。
- γ^{k-t}: 随着时间推移的折扣因子。误差发生得越晚（k 越大），其对当前时刻 t 的影响就被折扣得越厉害。

从上述推导我们可以看出，假设在整个回合中 V 是固定的，蒙特卡洛误差可以表示为**未来所有 TD 误差的折扣总和 (discounted sum of TD errors)**。

TD 预测的优势 (Advantages of TD prediction)

相比于动态规划 (DP) 和蒙特卡洛 (MC) 方法，TD 方法具有几个关键优势：
- **对比 DP：** TD 方法的一个明显优势是它们**不需要环境的完整知识或模型**（即不需要知道状态转移概率和奖励函数）。此外，在每一步中，TD 方法**不需要遍历整个状态空间**（扫掠状态空间往往计算量巨大）。
- **对比 MC：** TD 方法可以在模拟的**每一个时间步实时更新**估计值，而不需要等到整个回合结束。有些应用场景的回合非常漫长（甚至没有终点），如果把所有的学习都推迟到回合结束，通常会太慢。
- **实践表现：** 在实际应用中，人们通常发现 TD 方法比恒定步长 α 的蒙特卡洛方法**收敛得更快**。

重要概念：

- **Sample updates (采样更新):** 只根据智能体实际经历的“一步”经验（当前状态、动作、奖励、下一状态）来更新。它具有随机性，但计算成本极低。（TD 和 MC 属于此类）
- **Expected updates (期望更新):** 考虑从当前状态出发所有可能的后续状态，并根据概率进行加权平均更新。它非常精确，但需要环境模型且计算量大。（DP 属于此类）
- **TD error (δ_t):** 强化学习中最核心的信号之一。它衡量了“我刚刚收到的奖励加上我对未来状态的新预测”与“我之前对当前状态的预测”之间的**出入（Surprise / 意外程度）**。如果 $\delta_t > 0$，说明实际情况比预期的好；反之则比预期的差。
-

Sarsa

Algorithm 6.1 Sarsa algorithm

```
Initialize Q(s, a) for each s ∈ S and a ∈ A(s) arbitrarily
Set Q(s, a) = 0 for each terminal state s
for each episode do
    Choose S_0 arbitrarily
    Choose A_0 following a policy derived from Q (e.g. ε-greedy)
    for each time step t = 0, 1, ... do
        Take action a_t, observe r_{t+1} and s_{t+1}
        Choose a_{t+1} following a policy derived from Q (e.g. ε-greedy)
        Update Q(s_t, a_t) ← Q(s_t, a_t) + α (r_{t+1} + γQ(s_{t+1}, a_{t+1}) − Q(s_t, a_t)).
    end for
end for
return Q
```

算法伪代码逻辑

- 任意初始化所有状态-动作对的价值 $Q(s,a)$，其中 $s \in \mathcal{S}, a \in \mathcal{A}(s)$。
- 对于所有终止状态 (terminal state) s，将其价值设为 $Q(s,a) = 0$。
- **对每个回合 (episode) 进行循环：**
  - 任意选择一个初始状态 S_0。
  - 根据从 Q 推导出的策略（例如 ϵ-贪心策略），选择初始动作 A_0。
  - **对回合中的每个时间步 $t = 0, 1, \ldots$ 进行循环：**
    - 执行动作 a_t，观察收到的奖励 r_{t+1} 和下一个状态 s_{t+1}。
    - 再次根据从 Q 推导出的策略（例如 ϵ-贪心策略），在下一个状态 s_{t+1} 中选择下一个动作 a_{t+1}。
    - 更新 Q 值：$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\left(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)\right)$
  - **循环结束**

2. 算法核心说明

这种更新可以在每次从非终止状态 S_t 转移到新状态 S_{t+1} 后进行。如果 S_t 是终止状态，那么对于任何动作 A_t，$Q(S_t, A_t)$ 都被定义为零。

Sarsa 名称的由来： 这个更新规则使用了沿着轨迹发生的一个五元组事件：$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$。取这五个字母的首字母，就构成了算法的名字 **Sarsa**。

收敛性： Sarsa 算法以概率 1 收敛到最优策略和最优动作价值函数，前提是：

1. 所有的状态-动作对都被访问无限次。
2. 策略在极限情况下收敛为贪心策略（这可以通过逐渐将 ϵ 的值降低到 0 来实现）。

1. Sarsa 更新五元组：$(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$

- S_t, A_t: 当前的状态和在这个状态下选择的动作。
- R_{t+1}, S_{t+1}: 执行动作后，环境返回的即时奖励和转移到的下一个状态。
- A_{t+1}: 在下一个状态 S_{t+1} 中，根据**同样的策略**再次选择的下一个动作。Sarsa 必须真实地“看”到这第二步动作才能更新第一步。

期望 Sarsa

Q-learning 算法的一个变体。它不是基于实际选择的下一个动作 a_{t+1} 来估计 Q 值，而是计算**期望值 (expected value)**，即将当前策略下每个动作**被选择的可能性**也考虑进去。
修改后的更新规则如下：

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \mathbb{E}_\pi[Q(s_{t+1}, A_{t+1}) | s_{t+1}] - Q(s_t, a_t) \right) \quad (\text{公式 } 6.5)$$

$$= Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \sum \pi(a|s_{t+1}) Q(s_{t+1}, a) - Q(s_t, a_t) \right) \quad (\text{公式 } 6.6)$$

2. Expected Sarsa 期望公式符号：

- $\mathbb{E}_\pi[\dots]$: 在策略 π 下的**数学期望**。
- $\sum_a \pi(a|s_{t+1}) Q(s_{t+1}, a)$: 这是期望值的具体展开。
  - a: 代表在状态 s_{t+1} 下所有可能采取的动作。
  - $\pi(a|s_{t+1})$: 这是一个概率值，表示在状态 s_{t+1} 下，策略 π 选择动作 a 的概率。（例如，如果是 ϵ-贪心策略，最优动作的概率是 $1 - \epsilon + \epsilon/|\mathcal{A}|$，其他动作的概率是 $\epsilon/|\mathcal{A}|$）。
  - 这部分是将所有的 Q 值用它们被选择的概率进行了**加权平均**。它不需要像 Sarsa 那样实际去采样出 A_{t+1}，而是直接把所有可能性的平均值算出来作为目标。

给定下一个状态 S_{t+1}，这个算法在期望上与 Sarsa 移动的方向完全一致，并且是**确定性**地移动（因为它计算的是数学期望，消除了下一步动作选择带来的随机方差），因此被称为 **Expected Sarsa (期望 Sarsa)**。

在许多实际案例中，人们发现 Expected Sarsa 在很宽的步长参数 α 取值范围内，性能都显著优于普通的 Sarsa。

与 Q-learning 的关系：
一般来说，Expected Sarsa 可以使用与目标策略 π 不同的策略来生成行为（即与环境交互采样），在这种情况下，它变成了一个**异策略 (off-policy)** 算法。
特别地，如果目标策略 π 是一个纯贪心策略 (greedy policy)，而行为策略是更具探索性的策略（如 ϵ-贪心），那么**此时的 Expected Sarsa 就完全等价于 Q-learning 算法**。

核心小概念捏：

- **Action-Value Function (动作价值函数 Q 值):** 即 $Q(s,a)$。它评估的是在状态 s 下采取特定动作 a 的好坏程度，而不仅仅是状态本身的好坏（$V(s)$）。
- **On-policy (同策略) vs Off-policy (异策略):**
  • **Sarsa 属于 On-policy：** 它评估和优化的策略（目标策略）与它用来在环境中探索的策略（行为策略）是**同一个**策略（例如都是 ϵ-贪心）。它“言行一致”。
  • **Q-learning / (某些情况下的) Expected Sarsa 属于 Off-policy：** 它用来评估的策略（比如纯贪心策略，总是选最大 Q 值的动作）和它实际行动探索的策略（包含随机探索）是**分开的**。
- **ϵ-greedy (ϵ-贪心策略):** 为了保持探索 (exploration)，智能体在绝大多数时候（$1-\epsilon$ 的概率）选择当前 Q 值最大的最佳动作，但有一小部分时候（ϵ 的概率）会完全随机选择一个动作。

自举 (Bootstrapping)

通俗解释：
“Bootstrapping” 的字面意思是“提着自己的鞋带把自己拽起来”，这听起来很违背物理常识。在算法里，它的意思是指**“用一个猜测去更新另一个猜测”**，而不是等到最终的真实结果出来。

举例分析：
假设你在开发一个**量化交易策略**来预测某只股票年底的最终收益。

- **不自举（非 Bootstrapping）：** 你买入股票，然后什么都不管，硬生生等到 12 月 31 日收盘。看到赚了 20%，你才恍然大悟：“哦，原来我年初的买入策略价值是 +20%”。（这很慢，效率极低）

- **自举（Bootstrapping）：** 你今天买入股票，明天股票涨了 2%。你不需要等到年底，你直接用“今天的 2% 收益 + 我模型预测的未来收益”来**更新**你昨天对这只股票的估值。你是在用自己目前对未来的“估算值”，来倒推和修正当前的“估算值”。

这就是 TD 学习的核心：走一步看一步，用下一步的预测来修正这一步的预测，不需要等到游戏结束或年底清算。

控制问题 (Control Problem)

通俗解释：
在强化学习里，任务分为“预测（Prediction）”和“控制（Control）”。
• **预测：** 是评估“既定策略”有多好。比如，你闭着眼睛瞎按手柄，系统预测你存活时间只有 3 秒。
• **控制：** 是**寻找“最优策略”**。控制问题的终极目标是找到一种行为方式（Policy），让你能够拿到最高的分数、赚到最多的钱，或者最快地通关。
举例分析：
回到打 Boss 的场景。
“控制问题”就是你需要找到那套**绝对完美的连招和走位**（最优策略 π^*）。在什么距离该出重击？Boss 抬手时该往左滚还是往右滚？当你解决了控制问题，你就从一个总是挨打的新手，变成了一个能够无伤速通的大神。Sarsa 和 Q-learning 这些算法，本质上都是用来解决控制问题的。

广义策略迭代 (Generalized Policy Iteration, GPI)

通俗解释：
GPI 并不是一个具体的算法（比如不是几行代码），而是一个**宏观的概念框架**。几乎所有解决“控制问题”的强化学习算法都在使用 GPI 的思想。它描述了两个过程的互相交替、相辅相成：
1. **策略评估 (Policy Evaluation)：** 看看你现在的打法有多好/多烂。
2. **策略改进 (Policy Improvement)：** 根据刚才的评估结果，调整你的打法，让你变得更好。
举例分析：
想象你在不断优化你的量化交易机器人：
1. **策略评估：** 你先随便写了一套交易规则（当前策略），然后用历史数据回测，或者跑一周的模拟盘，记录下这套规则在各种行情下的盈亏情况（得出价值函数 V 或 Q）。
2. **策略改进：** 你看着回测报告发现：“只要遇到大阴线买入，后续大概率亏损”。于是你修改了规则，在遇到大阴线时选择空仓（将策略变为贪心策略）。
3. **循环往复：** 你用修改后的新规则再去跑回测（再次评估），然后再找漏洞修改规则（再次改进）。